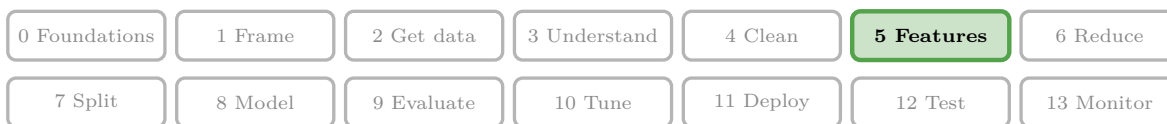


Handling Date and Time Variables

Contents

1. Overview	2
2. Why date and time columns are useful	2
3. The data	3
4. Converting text to a datetime type	4
5. Extracting parts of a date	5
5.1 Year	5
5.2 Month	5
5.3 Day of the month	5
5.4 Day of the week	6
5.5 Weekend or not	6
5.6 Week of the year	7
5.7 Quarter	8
5.8 Semester	8
6. Time passed since a date	8
6.1 Fixing “today”	8
6.2 The Timedelta	9
6.3 Months passed	10
7. Extracting parts of a time	11
8. Time passed in seconds, minutes and hours	12
9. Summary	14
Sources	14
10. Key terms	14

Where this fits: Pipeline map step 5 (Engineer features). See the [Course map](#).



- **Builds on:** CSV files ([Note 15](#)); Feature engineering ([Note 23](#)).

1. Overview

Key point: A date or time column hides many useful features (year, month, weekday, hour, time passed); we convert the column to a datetime type and pull each one out with `.dt`.

A single value such as 2019-12-10 23:40:00 looks like one piece of information. It actually holds a dozen: the year, the month, the weekday, whether it is a weekend, the hour, and more.

Figure 1 shows one value split into these features. The rest of the Note shows the pandas code for each one. The Notebook (`notebook.ipynb`) runs every example.

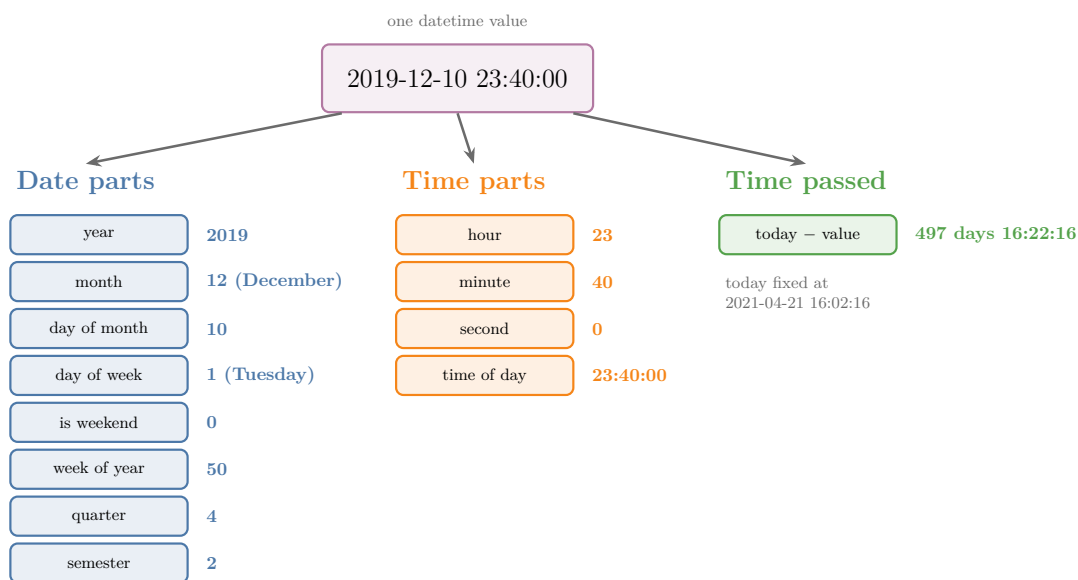


Figure 1: One datetime value and the features we can extract from it

2. Why date and time columns are useful

Key point: The hidden parts of a date (weekday, month, weekend) often explain behaviour better than the raw date does.

Take the date 23 August 2012. From it we can read:

- the day of the month (23), the month (August) and the year (2012);
- the day of the week (Thursday), and whether it is a weekend;
- the quarter of the year and the half of the year (semester).

A time such as 16:02:58 holds the hour, the minute and the second in the same way.

Consider an expense-tracker app that reports how a user spends money. The raw date of each payment says little. The extracted parts answer real questions:

- Does the user spend more on weekends than on weekdays?
- In which month does the user spend the most?
- At what time of day does most spending happen?

Each question needs one extracted feature: a weekend flag, the month, the hour. Building such columns is a kind of feature engineering.

3. The data

Key point: Two small files: `orders.csv` holds dates only, `messages.csv` holds dates with times.

We use two files of 1,000 rows each.

File	Columns	Range
<code>orders.csv</code>	date, product_id, city_id, orders	2018-07-10 to 2019-12-16
<code>messages.csv</code>	date (date and time)	2012-01-01 to 2017-01-18

`orders.csv` comes from an online shop: on which date, which product, in which city, how many orders. `messages.csv` gives the send time of 1,000 chat messages from a messaging app. The original file also holds the message text; we drop it, since only the times are used.

The first rows:

	date	product_id	city_id	orders
0	2019-12-10	5628	25	3
1	2018-08-15	3646	14	157
2	2018-10-23	1859	25	1
3	2019-08-17	7292	25	1
4	2019-01-06	4344	25	3

	date (messages)
0	2013-12-15 00:50:00
1	2014-04-29 23:40:00

	date (messages)
2	2012-12-30 00:21:00
3	2014-11-28 00:31:00
4	2013-10-26 23:11:00

4. Converting text to a datetime type

Key point: `read_csv` reads dates as text; `pd.to_datetime` converts them, and only then do the date tools work.

When pandas loads a CSV file, a date column arrives as plain text (dtype `str`). As text, 2019-12-10 is just ten characters: pandas cannot tell its month or weekday. So the first step is always to change the column's type.

`pd.to_datetime` converts text into **datetime** values: values that pandas understands as points in time.

Python: Converting the column.

```
import numpy as np
import pandas as pd

orders = pd.read_csv("data/orders.csv")
orders["date"].dtype      # str

orders["date"] = pd.to_datetime(orders["date"])
orders["date"].dtype      # datetime64[us]
```

	dtype of date	Date tools available?
Before	<code>str</code>	no
After	<code>datetime64[us]</code>	yes

The table itself looks the same before and after: only `orders.info()` shows the new type. The same conversion can be done while reading the file, with `parse_dates=["date"]` in `read_csv` (Note 15, Section 15).

Extra: The `[us]` in `datetime64[us]` is the **resolution**: the smallest step the column can store, here one microsecond. Older pandas always used nanoseconds and showed `datetime64[ns]`. Since pandas 3, text is parsed at microsecond resolution (pandas release notes, 3.0.0). The dates and every result in this Note are the same either way.

Extra: How `pd.to_datetime` reads the text. Since pandas 2 it guesses one format from the first value and applies it to every row (pandas release notes, 2.0.0, "Datetimes are now parsed with a consistent format"). That causes two surprises with day/month dates:

- "10/12/2019" is read month first, as 12 October 2019.
- ["10/12/2019", "25/12/2019"] stops with `ValueError: time data "25/12/2019" doesn't match format "%m/%d/%Y"`, because there is no month 25.

The fix is to state the format: `pd.to_datetime(s, format="%d/%m/%Y")` gives 10 and 25 December. Here `%d` is the day, `%m` the month and `%Y` the four-digit year. Two more options help with messy columns:

- `format="mixed"` guesses the format separately for each value.
- `errors="coerce"` turns a value that cannot be read into **NaT** (“not a time”), the datetime version of NaN.

5. Extracting parts of a date

Key point: `column.dt.<part>` returns one part of every date in the column at once.

The `.dt` accessor gives a datetime column its date tools, the way `.str` gives a text column its text tools. Each tool returns a new column with one value per row. We store each one as a new feature.

5.1 Year

Key point: `.dt.year` gives the year as a number.

Python: Year.

```
orders["date_year"] = orders["date"].dt.year
```

The orders cover two years: 355 rows from 2018 and 645 from 2019.

5.2 Month

Key point: `.dt.month` gives the month number (1 to 12); `.dt.month_name()` gives its name.

`month` is an attribute, written without brackets. `month_name()` is a method, written with brackets.

Python: Month number and name.

```
orders["date_month_no"] = orders["date"].dt.month
orders["date_month_name"] = orders["date"].dt.month_name()
```

5.3 Day of the month

Key point: `.dt.day` gives the day of the month (1 to 31).

Python: Day of the month.

```
orders["date_day"] = orders["date"].dt.day
```

The results so far:

	date	date_year	date_month_no	date_month_name	date_day
0	2019-12-10	2019	12	December	10
1	2018-08-15	2018	8	August	15
2	2018-10-23	2018	10	October	23
3	2019-08-17	2019	8	August	17
4	2019-01-06	2019	1	January	6

5.4 Day of the week

Key point: `.dt.dayofweek` numbers the days from Monday = 0 to Sunday = 6; `.dt.day_name()` gives the name.

The numbering starts at Monday, with 0, and ends at Sunday, with 6. So 10 December 2019, a Tuesday, gets 1.

Python: Day of the week.

```
orders["date_dow"] = orders["date"].dt.dayofweek
orders["date_dow_name"] = orders["date"].dt.day_name()
```

5.5 Weekend or not

Key point: A weekend flag is 1 for Saturday and Sunday and 0 for the other days.

There is no ready-made `.dt` tool for this, so we build it from the weekday. Saturday and Sunday are the only days numbered 5 or more.

Python: Weekend flag.

```
orders["date_is_weekend"] = (orders["date_dow"] >= 5).astype(int)
orders["date_dow"] >= 5 gives True or False for each row; .astype(int) turns True
into 1 and False into 0.
```

	date	date_dow	date_dow_name	date_is_weekend
0	2019-12-10	1	Tuesday	0
1	2018-08-15	2	Wednesday	0
2	2018-10-23	1	Tuesday	0
3	2019-08-17	5	Saturday	1
4	2019-01-06	6	Sunday	1

284 of the 1,000 orders fall on a weekend. Figure 2 (left) shows the rows per weekday.

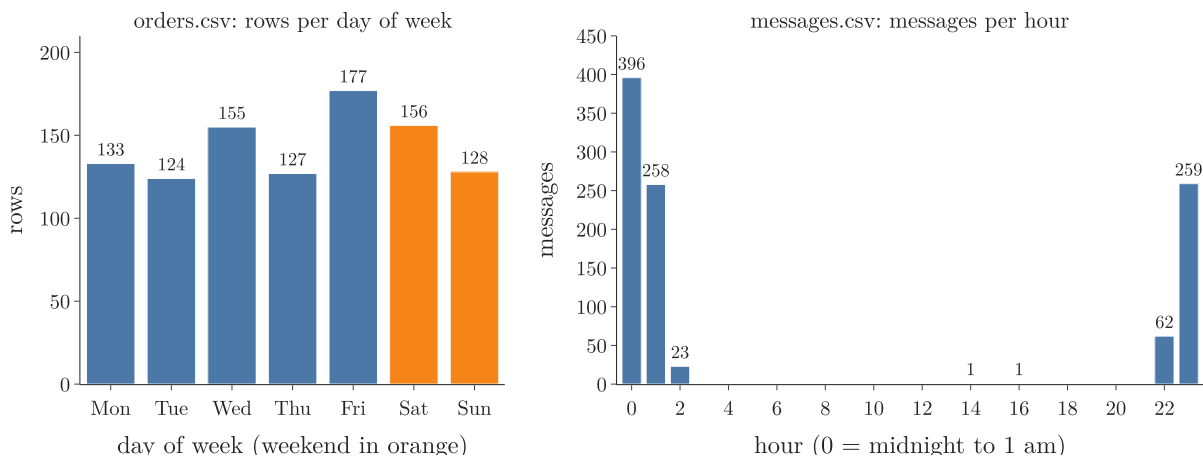


Figure 2: Left: rows of `orders.csv` per day of the week. Right: messages per hour of the day in `messages.csv`

Extra: Older code builds the same flag from the names: `np.where(orders["date_dow_name"].isin(["Saturday", "Sunday"]), 1, 0)`. `np.where(condition, a, b)` gives `a` where the condition is True and `b` elsewhere. Both ways give the same column; the number comparison does not depend on the names being in English.

5.6 Week of the year

Key point: `.dt.isocalendar().week` gives the week number (1 to 53); the older `.dt.week` no longer exists.

A year has 52 weeks, and sometimes 53. The week number tells us where in the year a date falls: 10 December 2019 is in week 50, 6 January 2019 in week 1.

Python: Week of the year.

```
orders["date_week"] = orders["date"].dt.isocalendar().week
.dt.isocalendar() returns a table with three columns, year, week and day; .week
takes the week column.
```

Older code writes `orders["date"].dt.week`. It was deprecated in pandas 1.1 and removed in pandas 2, so it now fails with `AttributeError: 'DatetimeProperties' object has no attribute 'week'`. The new way gives exactly the same numbers.

Extra: The week numbers follow the **ISO calendar**: weeks run Monday to Sunday, and week 1 is the week that contains the year's first Thursday (ISO 8601). So the last days of December can belong to week 1 of the next year. 29 December 2019 (a Sunday) is in week 52 of 2019, but 30 December 2019 (a Monday) is in week 1 of 2020. For such dates `isocalendar().year` differs from `.dt.year`.

5.7 Quarter

Key point: `.dt.quarter` splits the year into four parts of three months: January to March is quarter 1, and so on.

Python: Quarter.
`orders["quarter"] = orders["date"].dt.quarter`

December and October fall in quarter 4, August in quarter 3, January in quarter 1. Companies often report sales by quarter, so this feature matches how business data is usually grouped.

5.8 Semester

Key point: A semester is half a year: quarters 1 and 2 are semester 1, quarters 3 and 4 are semester 2.

pandas has no semester tool, so we build it from the quarter.

Python: Semester.
`orders["semester"] = np.where(
 orders["quarter"].isin([1, 2]), 1, 2)
 .isin([1, 2]) is True where the quarter is 1 or 2.`

	date	date_week	quarter	semester
0	2019-12-10	50	4	2
1	2018-08-15	33	3	2
2	2018-10-23	43	4	2
3	2019-08-17	33	3	2
4	2019-01-06	1	1	1

6. Time passed since a date

Key point: Subtracting two datetimes gives a `Timedelta`; dividing it by a unit turns it into a number of days, hours or months.

How long ago an order was placed is often a useful feature, for example “days since the last purchase”. We get it by subtracting the date from today’s date.

6.1 Fixing “today”

Key point: We write today’s date as a fixed value, so the results stay the same whenever the code runs.

`pd.Timestamp.now()` (or `datetime.datetime.today()`) returns the moment the code runs, so every run gives different numbers. In this Note “today” is fixed at 21 April 2021, 16:02:16.

Python: A fixed “today”.

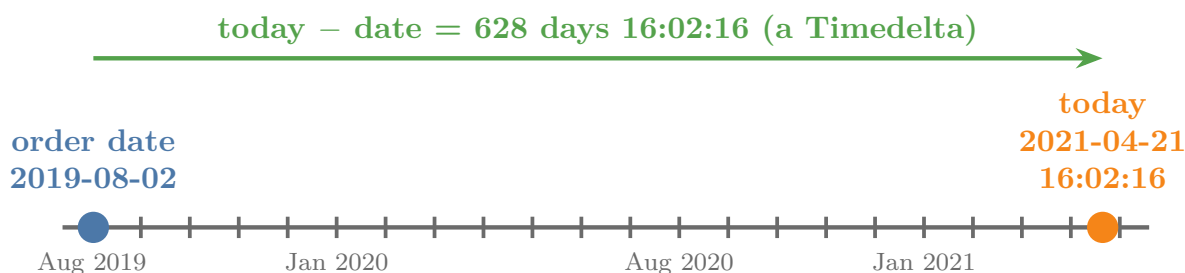
```
today = pd.Timestamp("2021-04-21 16:02:16")
```

A **Timestamp** is pandas’ type for one single point in time; a datetime column is a column of Timestamps.

6.2 The Timedelta

Key point: `today - orders["date"]` gives one Timedelta per row: a length of time in days, hours, minutes and seconds.

A **Timedelta** is a length of time, such as 498 days 16:02:16. Subtracting one Timestamp from another gives one. Figure 3 shows it for one order.



in days: $628 \text{ days } 16:02:16 \div 1 \text{ day} = 628.67$

in months, average month: $\div 30.44 \text{ days} = 20.65$, rounded to **21**

in months, calendar: Aug 2019 $\rightarrow$ Apr 2021 = **20** month numbers apart

Figure 3: The time between an order and “today”, and three ways to turn it into a number

Python: Time passed.

```
elapsed = today - orders["date"]
```

```
elapsed.dt.days
```

whole days

```
elapsed / np.timedelta64(1, "D")
```

days, with fractions

	date	elapsed	.dt.days	/ 1 day
0	2019-12-10	498 days 16:02:16	498	498.668
1	2018-08-15	980 days 16:02:16	980	980.668
2	2018-10-23	911 days 16:02:16	911	911.668
3	2019-08-17	613 days 16:02:16	613	613.668
4	2019-01-06	836 days 16:02:16	836	836.668

There are two ways to get days:

- `.dt.days` keeps only the whole days and drops the hours.
- **Dividing by one day**, `np.timedelta64(1, "D")`, keeps the fraction: 16 hours 2 minutes is 0.668 of a day.

`np.timedelta64(1, "D")` is NumPy's way of writing a Timedelta of one day. `pd.Timedelta(days=1)` means the same.

6.3 Months passed

Key point: A month has no fixed length, so “months passed” must be computed with an average month or by counting calendar months.

To get months, older code divides by `np.timedelta64(1, "M")`, one month. Current pandas refuses: `ValueError: Unit M is not supported. Only unambiguous timedelta values durations are supported.` A month can be 28, 29, 30 or 31 days long, so “one month” is not a fixed length of time.

Way 1: divide by an average month. In words: the months passed are the days passed divided by the length of an average month, rounded to a whole number.

$$\text{months} = \text{round} \left(\frac{\text{days passed}}{30.436875} \right)$$

The 30.436875 days are one year of 365.2425 days, the average length of a calendar year, divided by 12. This is exactly the length `np.timedelta64(1, "M")` used to stand for.

For row 0, ordered on 10 December 2019:

$$\frac{498.668}{30.436875} = 16.38 \rightarrow 16 \text{ months}$$

Way 2: count calendar months. Subtract the month numbers, counting 12 for each year in between.

$$\text{months} = 12 \times (\text{year}_{\text{today}} - \text{year}_{\text{date}}) + (\text{month}_{\text{today}} - \text{month}_{\text{date}})$$

For row 0: $12 \times (2021 - 2019) + (4 - 12) = 24 - 8 = 16$ months.

Python: Months passed, both ways.

way 1: average month length

```
avg = pd.Timedelta(days=30.436875)
months_avg = (elapsed / avg).round()
```

way 2: calendar months

```
d = orders["date"]
months_cal = ((today.year - d.dt.year) * 12
```

```
+ (today.month - d.dt.month))
```

	date	average month	calendar months
0	2019-12-10	16	16
1	2018-08-15	32	32
2	2018-10-23	30	30
3	2019-08-17	20	20
4	2019-01-06	27	27
11	2019-08-02	21	20

The two ways agree on the first five rows, but disagree on 205 of the 1,000. Row 11 shows why (Figure 3): from 2 August 2019 to 21 April 2021 is 20 months and 19 days. The average-month way gives 20.65 and rounds it up to 21; the calendar way counts 20.

Extra: Neither way is wrong; they answer slightly different questions. The average-month way measures a length of time, the calendar way counts month changes. Pick one and use it for every row. For a model, days passed (`.dt.days`) is often the simplest choice: it is exact and needs no rounding.

7. Extracting parts of a time

Key point: With a time in the column, `.dt.hour`, `.dt.minute` and `.dt.second` work just like the date parts.

`messages.csv` holds dates with times. The conversion is the same `pd.to_datetime`, and the time parts come from the same `.dt` accessor.

Python: Hour, minute, second and time of day.

```
messages = pd.read_csv("data/messages.csv")
messages["date"] = pd.to_datetime(messages["date"])
```

```
messages["hour"] = messages["date"].dt.hour
messages["min"] = messages["date"].dt.minute
messages["sec"] = messages["date"].dt.second
messages["time"] = messages["date"].dt.time
```

	date	hour	min	sec	time
0	2013-12-15 00:50:00	0	50	0	00:50:00
1	2014-04-29 23:40:00	23	40	0	23:40:00
2	2012-12-30 00:21:00	0	21	0	00:21:00
3	2014-11-28 00:31:00	0	31	0	00:31:00
4	2013-10-26 23:11:00	23	11	0	23:11:00

`.dt.time` keeps only the time of day and drops the date. It is useful when only the clock time matters.

The hour feature already tells a story. Figure 2 (right) shows that 998 of the 1,000 messages were sent between 22:00 and 03:00: this is a night-time chat app.

Extra: `.dt.time` returns a column of Python `datetime.time` objects (dtype `object`), not a datetime column. The `.dt` tools no longer work on it, and a model cannot use it directly. For a model, the `hour` and `min` numbers are the better features.

8. Time passed in seconds, minutes and hours

Key point: Divide the Timedelta by one second, one minute or one hour to get the time passed in that unit.

The subtraction works as in Section 6, now with times included. Only the unit we divide by changes.

Python: Time passed in three units.

```
since = today - messages["date"]
since / np.timedelta64(1, "s")    # seconds
since / np.timedelta64(1, "m")    # minutes
since / np.timedelta64(1, "h")    # hours
```

	elapsed	seconds	minutes	hours
0	2684 days 15:12:16	231,952,336	3,865,872.3	64,431.20
1	2548 days 16:22:16	220,206,136	3,670,102.3	61,168.37
2	3034 days 15:41:16	262,194,076	4,369,901.3	72,831.69
3	2336 days 15:31:16	201,886,276	3,364,771.3	56,079.52
4	2733 days 16:51:16	236,191,876	3,936,531.3	65,608.85

Check row 0 in hours: 2684 days are $2684 \times 24 = 64,416$ hours, and 15:12:16 adds 15.20 hours, so 64,431.20 hours.

The unit letters are case-sensitive:

Letter	Unit
"D"	day
"h"	hour
"m"	minute
"s"	second
"M"	month: no longer allowed (Section 6.3)

Extra: `since.dt.total_seconds()` gives the same seconds without NumPy. Dividing by `pd.Timedelta(hours=1)` or `pd.Timedelta(minutes=1)` works too, and reads more clearly than the single letters.

Extra: Dates and times can also carry a **time zone**, such as India's +05:30. Mixing values from different time zones needs extra care (`.dt.tz_localize` and `.dt.tz_convert`). This is rarely needed in ML datasets; our two files have no time zone.

9. Summary

Feature	Code	Row 0 of orders
Convert text	<code>pd.to_datetime(col)</code>	2019-12-10
Year	<code>.dt.year</code>	2019
Month	<code>.dt.month,</code> <code>.dt.month_name()</code>	12, December
Day of month	<code>.dt.day</code>	10
Day of week	<code>.dt.dayofweek,</code> <code>.dt.day_name()</code>	1, Tuesday
Weekend	<code>(dayofweek >=</code> <code>5).astype(int)</code>	0
Week of year	<code>.dt.isocalendar().week</code>	50
Quarter, semester	<code>.dt.quarter, from the</code> <code>quarter</code>	4, 2
Days passed	<code>(today - col).dt.days</code>	498
Months passed	<code>/</code> <code>pd.Timedelta(days=30.436875),</code> <code>rounded</code>	16
Hour, minute, second	<code>.dt.hour, .dt.minute,</code> <code>.dt.second</code>	(messages) 0, 50, 0

- Dates arrive as text; convert them with `pd.to_datetime` (or `parse_dates` in `read_csv`) before anything else.
- The `.dt` accessor pulls out each part of a date or time as a new feature.
- Weekend and semester flags are built from the weekday and the quarter.
- `.dt.week` was removed: use `.dt.isocalendar().week`.
- Subtracting datetimes gives a `Timedelta`; divide it by a unit to get a number.
- A month has no fixed length: use an average month of 30.436875 days, or count calendar months.
- Fix “today” as a constant so that results do not change between runs.
- Give `pd.to_datetime` a `format` when dates are written day first.

Sources

- ISO 8601. Date and time format, week dates. International Organization for Standardization.
- pandas release notes. What’s new in 2.0.0 and 3.0.0. pandas.pydata.org/docs/whatsnew.

10. Key terms

Term	Meaning
Datetime	A value pandas understands as a point in time, with date and time parts
<code>pd.to_datetime</code>	The pandas function that converts text to datetime values
<code>datetime64</code>	The pandas column type for datetimes; <code>[us]</code> means microsecond resolution
<code>.dt</code> accessor	The pandas tool that applies date and time methods to every value of a datetime column
Day of week	The weekday as a number, Monday = 0 to Sunday = 6 (<code>.dt.dayofweek</code>)
ISO week	The week number of the ISO calendar, from <code>.dt.isocalendar().week</code> ; week 1 holds the year's first Thursday
Quarter	One of four three-month parts of a year
Semester	One of two six-month halves of a year
Timestamp	pandas' type for a single point in time
Timedelta	A length of time, the result of subtracting two datetimes
NaT	"Not a time": the missing value of a datetime column
Format string	A pattern such as <code>"%d/%m/%Y"</code> telling <code>pd.to_datetime</code> how dates are written